

Interfejsy w języku C#

Interfejs to specjalny typ w C#, który definiuje **zestaw metod, właściwości lub zdarzeń**, ale **nie zawiera ich implementacji** (czyli nie ma ciała metod).

Interfejs mówi *co klasa powinna robić*, ale **nie jak** ma to robić.

Można powiedzieć, że interfejs to **umowa** — jeśli klasa deklaruje, że go implementuje, to musi dostarczyć wszystkie metody i właściwości, które interfejs określa.

Składnia interfejsu

```
interface INazwaInterfejsu
{
    void Metoda1();
    int Wlasciwosc { get; set; }
}
```

Zasady:

- Interfejsy **zaczynamy od litery I** (np. IAnimal, IDrukowalne).
- Wszystkie elementy interfejsu są **domyślnie publiczne i abstrakcyjne** — nie używamy słów public ani abstract.
- Klasa, która implementuje interfejs, musi zaimplementować **wszystkie** jego elementy.

Przykład prostego interfejsu

```
interface IDrukowalne
{
    void Drukuj();
}

class Dokument : IDrukowalne
{
    public void Drukuj()
    {
        Console.WriteLine("Drukuję dokument...");
    }
}

class Zdjecie : IDrukowalne
{
    public void Drukuj()
    {
        Console.WriteLine("Drukuję zdjęcie...");
    }
}
```

```
class Program
{
    static void Main()
    {
        List<IDrukowalne> elementy = new List<IDrukowalne>
        {
            new Dokument(),
            new Zdjecie()
        };

        foreach (var e in elementy)
            e.Drukuj();
    }
}
```

Wynik:

Drukuję dokument...
Drukuję zdjęcie...

To przykład **polimorfizmu interfejsów** — różne klasy implementują ten sam interfejs, więc można je przechowywać w jednej kolekcji.

Klasa może implementować wiele interfejsów

C# nie pozwala na **dziedziczenie wielokrotne klas**, ale pozwala na **implementację wielu interfejsów**.

```
interface IStartable
{
    void Start();
}

interface IStoppable
{
    void Stop();
}

class Samochod : IStartable, IStoppable
{
    public void Start() => Console.WriteLine("Silnik uruchomiony!");
    public void Stop() => Console.WriteLine("Silnik zatrzymany.");
}
```

Zalety:

- umożliwia łączenie różnych „umów” w jednej klasie,
- zwiększa elastyczność projektu,
- wspiera zasadę *kompozycji zamiast dziedziczenia*.

Interfejsy a klasy abstrakcyjne – różnice

Cecha	Klasa abstrakcyjna	Interfejs
Dziedziczenie	Tylko jedna klasa bazowa	Można implementować wiele interfejsów
Pola	Może zawierać pola	Nie może zawierać pól (tylko właściwości i metody)
Implementacja metod	Może zawierać implementację	Nie zawiera implementacji (z wyjątkiem domyślnej od C# 8)
Konstruktor	Może mieć konstruktor	Nie ma konstruktorów
Słowo kluczowe	abstract class	interface

Interfejsy w praktyce – przykład z pojazdami

```
interface IPojazd
{
    void Uruchom();
    void Zatrzymaj();
}

class Samochod : IPojazd
{
    public void Uruchom() => Console.WriteLine("Samochód rusza.");
    public void Zatrzymaj() => Console.WriteLine("Samochód hamuje.");
}

class Rower : IPojazd
{
    public void Uruchom() => Console.WriteLine("Rower zaczyna jechać.");
    public void Zatrzymaj() => Console.WriteLine("Rower się zatrzymuje.");
}

class Program
{
    static void Main()
    {
        List<IPojazd> pojazdy = new List<IPojazd>
        {
            new Samochod(),
            new Rower()
        };

        foreach (var p in pojazdy)
        {
            p.Uruchom();
            p.Zatrzymaj();
        }
    }
}
```

Polimorfizm interfejsów

Interfejs pozwala traktować różne klasy w taki sam sposób, jeśli implementują ten sam zestaw metod. To **najczystsza forma polimorfizmu**.

```
IPojazd pojazd = new Samochod();  
pojazd.Uruchom(); // wywoła metodę z klasy Samochod
```

Podsumowanie

- ◆ **Interfejs** to umowa – mówi „co” klasa ma robić, ale nie „jak”.
- ◆ Klasa może implementować **dowolną liczbę interfejsów**.
- ◆ Interfejsy są podstawą **polimorfizmu i elastycznego projektowania**.
- ◆ W projektach rzeczywistych interfejsy często używane są do **testowania, wstrzykiwania zależności (Dependency Injection)** i budowy skalowalnych aplikacji.